

Woodshop Access Control – System Documentation

Oct 6, 2026 · @bruce

1. Overview

A member can run a woodshop machine only if they are signed in at the door kiosk and their permissions include that machine. Three kinds of device cooperate to enforce this.

Part	What it is	Role
Machine client ("client", "node")	One ESP32 per machine, Arduino firmware	Reads the RFID card, switches the machine and blast-gate relays, measures current, reports to the server
Server	Raspberry Pi 2B running DietPi, address 192.168.0.5	<code>master_server.py</code> (machine and kiosk TCP listeners, SQLite, LEDs) and <code>app.py</code> (web pages on port 80)
Kiosk ("Login")	Lee Robertshaw's Raspberry Pi at the door	Sends a LOGIN or LOGOUT message to the server when a member signs in or out

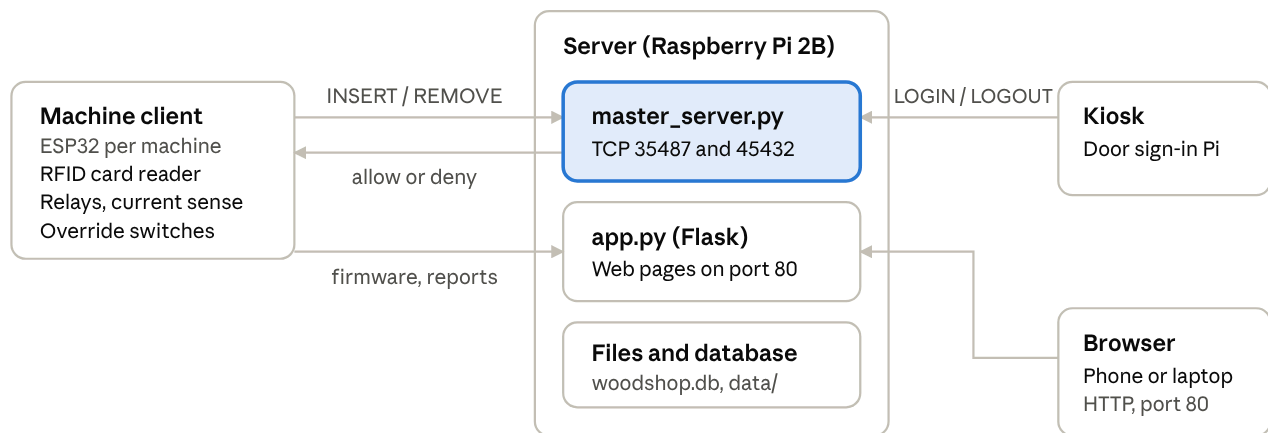
Typical sequence

1. The member signs in at the kiosk. The server adds a row to its active-members table.
2. The member taps their NTAG215 card on a machine. The client reads the card and switches the machine on straight away, using the permissions stored on the card.
3. The client sends an INSERT message (machine number, member ID) to the server.
4. The server checks that the member is signed in, then looks up their permission for that machine in the member file. It replies with status 0 (grant) or a non-zero status (deny).
5. On a deny the client clears its authorization flag, the relay drops, and the machine-on time for that session is reported as 0.

6. When the card is removed the client sends a REMOVE message with machine-on time, average current and start/stop counts. The server records the session and appends a row to the access log.
7. The member signs out at the kiosk; the server deletes their active-members row.

The web app shows who is signed in, the access log, member and machine administration, and daily log files for download.

One Pi decides access for every machine and serves the web pages



system overview · 3 device types, 1 server

Machines send INSERT and REMOVE messages and get an allow-or-deny reply. The kiosk sends sign-in and sign-out messages. Browsers, and the clients' firmware and diagnostics traffic, use the web app on port 80.

2. Network interfaces and message formats

The server listens on three TCP ports; all devices sit on the shop WiFi network (SSID set in the client's `config.h`). The client finds the server at the same subnet with last octet 5, so 192.168.0.5.

Port	Protocol	Used by	Handler
35487	Binary TCP, 16-byte request, 8-byte reply, one connection per message	Machine clients	<code>WoodshopServer</code> in <code>master_server.py</code>

Port	Protocol	Used by	Handler
45432	Binary TCP, 45-byte message, no reply, one connection per message	Kiosk (Login)	LoginListener in master_server.py
80	HTTP (Flask)	Browsers; clients for firmware and diagnostics	app.py

Machine to server: 16 bytes, big-endian (port 35487)

Offset	Size	Field	Notes
0	1	Machine number	1 to 128; stored on the client in flash, set by config card
1	4	Member ID	From the card
5	2	Machine-on time (s)	Time current was flowing; 0 on INSERT
7	2	Average current	Units of 0.01 A
9	4	Connect time (s)	How long the card was in; 0 on INSERT
13	1	Override flags	0x01 main override, 0x02 blast override, 0x40 watchdog fault
14	1	Starts	Machine start count
15	1	Stops	Machine stop count

There is no event-type field. The server infers it: all of duration, connect time, starts and stops equal to 0 is an INSERT (card in); any of them above 0 is a REMOVE (card out); override bit 0x40 is a FAULT. A card held for under one second therefore looks like an INSERT.

Server to machine: 8 bytes, big-endian

Offset	Size	Field
0	1	Machine number (echo)

Offset	Size	Field
1	4	Member ID (echo)
5	2	Status code
7	1	Update available (1 = firmware update ready)

Status	Name	Meaning
0x0000	OK	Access granted / message recorded
0x0001	ERROR	Generic server error
0x0002	MEMBER_NOT_FOUND	Reserved
0x0003	MACHINE_DISABLED	Reserved
0x0004	MEMBER_NOT_AUTHORIZED	Not signed in, inactive or expired, or no permission for this machine
0x0005	INVALID_MESSAGE	Short or malformed message

The client treats any non-zero status on an INSERT as a denial.

Kiosk to server: 45 bytes, big-endian (port 45432)

Offset	Size	Field	Notes
0	1	Message type	0 = login, 1 = logout
1	8	Timestamp	Unix epoch seconds, UTC
9	4	Member ID	Integer
13	16	First name	ASCII, null-padded
29	16	Last name	ASCII, null-padded

This format was confirmed with Lee Robertshaw on August 19, 2026.

`BruceTestSender.py` is the reference sender. An earlier 52-byte ASCII draft (`esp32_client.py`) is obsolete.

HTTP endpoints used by the clients

Endpoint	Purpose
GET /firmware/version, GET /firmware/manifest.json	Firmware version check
GET /firmware/<file>	Download firmware.bin (white LED on the server lights during a download)
POST /diag/report	Reboot diagnostics posted by a client after a reset

3. Server: master_server.py

master_server.py is the access-control engine. It runs as one process with several threads and writes everything to master.log and the console.

Thread	Job
WoodshopServer (port 35487)	Receives machine INSERT, REMOVE and FAULT messages, decides access, replies
LoginListener (port 45432)	Receives kiosk LOGIN and LOGOUT messages, maintains the signed-in list
daily-tasks	2:00 AM log backup and 4:00 AM NTP sync (section 6)
Main thread	Sets the status LEDs and logs a [HEARTBEAT] alive threads=N line every 60 s

How access is decided

When a machine sends an INSERT, the server replies 0x0004 (not authorized) unless **all** of these are true:

1. The member has a row in active_members (signed in at the kiosk).
2. The member exists in data/users.json, is marked active, and has not expired. A membership counts as current until its expiry date plus 90 days (MEMBERSHIP_GRACE_DAYS, the same rule the web app uses).

3. The permission string for the member has a `1` in the character at index machine number minus 1. Bit 0, the first character, is machine 1.

The permission string is re-read from `users.json` on every request, so edits take effect immediately. A message with the override bit (0x01) set skips these checks. If authorized, the server records a session unless one is already open for that member and machine, which makes a repeated INSERT harmless.

On a REMOVE the server closes the session in the database and appends a row to `access_log.csv`. A REMOVE with no matching INSERT, which is what a denied card produces, still creates a row; its start time is blank.

Kiosk messages

A LOGIN looks the member up in `users.json` and inserts or replaces the `active_members` row (name from the message, login time from the message timestamp converted to server local time). A LOGOUT deletes the row. Both also append a `LOGIN` or `LOGOUT` row to `access_log.csv` so they appear on the web Logs page.

Debug trace

`LOGIN_DEBUG = True` (above the `LoginListener` class) logs `[LOGIN-DBG]` lines for each kiosk message: the connection, raw bytes in hex, any extra bytes, the parsed fields, and the resulting `active_members` row. Set it to `False` to quiet the log.

Status LEDs on the Pi

GPIO	Colour	Meaning
25	Red	Server inoperable (a listener thread stopped or an exception escaped)
20	Green	Server running normally
5	Blue	Flashes 1 s on each kiosk message (port 45432)
26	Yellow	Flashes 1 s on each machine message (port 35487)
27	White	A client is downloading firmware (driven by <code>app.py</code>)

If `gpiozero` or its pin backend is missing, the server logs a warning and runs without LEDs.

Other points

- `UPDATE_AVAILABLE` at the top of the file is sent to every client in byte 7 of each reply (section 10 covers how clients react).
- The `MACHINES` table that gives machine numbers their names in logs is 0-based (0 = Table Saw). Permission bits are 1-based. See section 12.
- The header comment at the top of `master_server.py` still describes an older message layout (event-type and auth-status bytes). The parser itself uses the layout in section 2.

4. Server data stores

Everything lives under the server's install folder (`~/woodshop`). Back up the whole folder, or at minimum `data/` and `woodshop.db` .

Path	Written by	What it holds
<code>woodshop.db</code>	<code>master_server.py</code>	SQLite database (tables below). Recreated empty if deleted
<code>data/users.json</code>	<code>app.py</code> (admin pages, CSV import)	Member file: id, first and last name, email, phone, rfid, active flag, joined and expiry dates, 128-character permission string
<code>data/machines.json</code>	<code>app.py</code>	Machine list: id, name, location, enabled
<code>data/admin_creds.json</code>	<code>app.py</code>	Web login: username and SHA-256 password hash. Created as <code>admin / woodshop</code> if missing, so change it
<code>data/access_log.csv</code>	<code>master_server.py</code>	The log the web Logs page reads (formats below)

Path	Written by	What it holds
<code>data/logs/log<yyyy><MMM><dd>.csv</code>	<code>master_server.py</code> daily job	One backup file per day, for example <code>log2026OCT05.csv</code>
<code>firmware/firmware.bin</code> , <code>firmware/version.txt</code>	You	Firmware image and its version string served to the clients
<code>master.log</code>	Both programs	Text log. Keep one process per log file in mind: a running program keeps writing to a deleted log until restarted
<code>data/active_sessions.json</code>	none	Legacy, unused since August 2026

If `users.json` or `machines.json` is missing when the web app starts, it creates demo data (Alice Smith, Bob Jones, Charlie Brown and eight sample machines) and a sample `access_log.csv`. Import your real members before relying on the log.

SQLite tables (woodshop . db)

Table	Purpose
active_members	Who is signed in at the kiosk: member_id (key), first and last name, login_time, permissions snapshot. This is what the Active page shows and what the access check consults first
sessions	One row per machine session: member, machine, insert and remove times, machine-on seconds, average amps, connect seconds, override flags
machines	Current state per machine number: who is using it, session start, last seen, status
raw_messages	Every machine message received, with its hex dump, for troubleshooting
members	Created but not used by the current code

access_log.csv row formats

Row kind	Columns
Machine session (11)	timestamp (stop time), member_id, member_name, machine_number, machine_name, start_time, stop_time, connect_time_s, machine_on_s, avg_current_A, starts
Kiosk event (6)	timestamp, member_id, member_name, machine_number (blank), machine_name ("Kiosk"), event (LOGIN or LOGOUT)

The daily backup files use one 12-column layout: the 11 session columns plus an event column, with kiosk rows widened to fit. A session row with a blank start time is a REMOVE that had no matching INSERT, which is what a denied card produces.

5. Server: web app (app.py)

`app.py` is a Flask app on port 80, used from a phone, tablet or laptop on the shop WiFi at `http://192.168.0.5`. Every page except the client endpoints needs the admin login. The top menu on every page reads **Refresh, Logs, Log Files, Active, Users, Machines, Renew, Diagnostics, Logout**.

Menu item	Route	What it does
Refresh	none (browser reload)	Reloads the page you are on with current data; never changes page. Pages are sent with no-cache headers so a reload always fetches fresh data
Logs	<code>/logs</code>	The access log, newest first, with filters for machine, user, date range and last N rows. Shows machine sessions and kiosk LOGIN and LOGOUT rows. Event badges: SESSION, LOGIN, LOGOUT
Log Files	<code>/logfiles</code>	Lists the daily CSV backups, newest first, with date and size; click a name to download
Active	<code>/active</code>	Members currently signed in at the kiosk, with sign-in time and the machines they may use. Reloads itself every 30 s
Users	<code>/admin/users</code>	Member list, search, add, edit, delete, CSV import and export
Machines	<code>/admin/machines</code>	Machine list and editing, CSV import and export
Renew	<code>/admin/renew</code>	Bulk membership renewal: tick the members who paid, choose the year, and their expiry becomes Dec 31 of that year
Diagnostics	<code>/admin/diag</code>	Reboot reports posted by the clients
Logout	<code>/logout</code>	Ends the session

Other routes: `/admin/password` (change the admin password), JSON at `/api/logs` and `/api/active`, CSV export at `/api/users/export.csv`, `/api/machines/export.csv` and `/api/log/export.csv`, CSV import at `/api/users/import.csv` and `/api/machines/import.csv`.

Members and permissions

A member's `permissions` value is a 128-character string of `0` and `1`; character 1 is machine 1. In the CSV import the columns are `member_id`, `first_name`, `last_name`, `email`, `phone`, `rfid`, `joined`, `expiry`, `active`, `permissions`. Import merges: rows replace members with the same ID and everyone else is kept. A blank permissions value becomes all zeros (no access).

A membership is current while the member is marked active and today is no later than the expiry date plus 90 days. The Users page colours members by this status, and the server applies the same rule when it decides machine access.

Client endpoints (no login)

`/firmware/version`, `/firmware/manifest.json`, `/firmware/<file>` and `POST /diag/report` are called by the machine clients, so they do not require the admin login. Keep the network private.

6. Server: daily jobs and log files

A background thread in `master_server.py` runs two jobs once per day at local time. The times, NTP host and folder are constants in the block headed "Daily maintenance" (`LOG_BACKUP_TIME`, `NTP_SYNC_TIME`, `NTP_SERVER`, `LOG_BACKUP_DIR`).

Time	Job	Result
2:00 AM	Log backup	Writes the previous day's rows from <code>access_log.csv</code> to <code>data/logs/log<yyyy><MMM><dd>.csv</code> . The original log is left untouched. Days with no activity still get a header-only file
4:00 AM	NTP sync	Reads the time from <code>pool.ntp.org</code> . If the clock is off by more than 1 second it sets the system clock; otherwise it only logs the offset

File names. `log2026OCT05.csv` holds October 5, 2026. The month is a three-letter upper-case code. Files are written to a temporary name and renamed, so a partial file never appears.

Start-up back-fill. When the server starts it creates yesterday's file if it is missing, for example after a power cut at 2:00 AM.

Setting the clock needs permission. The server runs as the `woodshop` user. To let it set the time, add this line with `sudo visudo -f /etc/sudoers.d/woodshop-date`:

```
woodshop ALL=(root) NOPASSWD: /bin/date
```

Without it the job still checks the offset and logs a warning, but cannot change the clock. If DietPi's own time sync is active, offsets are normally tiny. The Pi 2B has no battery clock, so the time is wrong after a power-up until a sync happens.

Log lines to look for in `master.log`: `[BACKUP] wrote N row(s) for <date> -> <path>`, `[NTP] clock OK (offset ...)`, and `[NTP] clock was ... off -- set from ...`.

Downloading. The web app's Log Files page lists the files in `data/logs` newest first. Only names of the exact form `log<yyyy><MMM><dd>.csv` can be downloaded.

7. Server operations

Programs. Two separate programs run on the Pi: `app.py` (web, port 80) and `master_server.py` (ports 35487 and 45432). The code comments name them as the systemd services `woodshop` and `woodshop-tcp`. Both use the Python in `~/woodshop/venv`.

Restart after copying a new file.

```
sudo systemctl restart woodshop      # app.py
sudo systemctl restart woodshop-tcp  # master_server.py
```

If you start them by hand instead, stop the old process first (`pkill -f master_server.py`) and run `./venv/bin/python master_server.py` or `app.py` from `~/woodshop`. Run `ps aux | grep -E "master_server|app.py" | grep -v grep` to confirm exactly one of each is running; an old process left behind keeps serving the old code.

Watching activity.

```
tail -f ~/woodshop/master.log | grep --line-buffered "LOGIN"    # kiosk
messages
grep -E "NOT AUTHORIZED|INSERT:|Response:" ~/woodshop/master.log | tail -20
# machine decisions
```

Reading a machine decision. The `Response:` line shows `Status=0x0000` for a grant and `Status=0x0004` for a denial. The `Auth=AUTHORIZED` text on the `From ...: INSERT` line is a fixed default from the parser and does not reflect the decision.

Resetting the data. Stop both programs first. Then remove the files you want cleared: `master.log`, `woodshop.db`, and `data/access_log.csv` (replace the last with an empty file, `> data/access_log.csv`, so the web app does not re-create demo rows). Keep `data/users.json`, `data/machines.json` and `data/admin_creds.json` unless you mean to lose members, machines and the admin password.

Stale sign-ins. A kiosk LOGIN with no matching LOGOUT leaves the member signed in. Inspect with `sqlite3 ~/woodshop/woodshop.db "SELECT * FROM active_members;"` and clear one with `DELETE FROM active_members WHERE member_id=N;`.

Troubleshooting.

Symptom	Check
Kiosk message light flashes but nothing happens	<code>tail master.log</code> ; confirm the running copy has the 45-byte parser (startup line says "expecting 45-byte binary messages")
Member signed in but machine denies	Permission string in <code>users.json</code> has no <code>1</code> at machine number minus 1, member is inactive or expired, or the machine number does not match
Machine runs although denied	The server replied <code>0x0004</code> but the client firmware is older than the version that acts on it; update the client
Log page looks old	Press Refresh; the Active page reloads itself, Logs does not
Logs missing after a clean-up	A running program kept the deleted file open; restart it
Clock wrong after power-up	Section 6: NTP sync runs once a day at 4:00 AM only

8. Client: hardware and firmware structure

Each machine has one ESP32 board running the sketch `Client.ino` (Arduino IDE 2.3.10, board "ESP32 Dev Module"). Firmware version is `FW_VERSION` in `config.h` (1.0.5 at the time of writing). Libraries: Elechouse PN532 (`PN532` , `PN532_SPI`), rweather Crypto (Ed25519), ArduinoJson. The Tools > Partition Scheme must be one of the "with OTA" schemes or firmware updates cannot be installed.

Pins (`config.h`)

Function	GPIO	Notes
PN532 SS (chip select)	21	10 k pull-up to 3V3
PN532 MOSI / MISO / SCK	19 / 23 / 18	MISO and MOSI are the opposite of the ESP32 defaults; do not "fix" them
PN532 IRQ	4	Wired; interrupt currently not attached
PN532 reset (RSTPD_N)	13	Driven low 150 ms to reset the reader
LEDs red / green / yellow / white	12 / 14 / 27 / 26	Active high
Main relay / blast-gate relay / aux relay	32 / 33 / 25	Active high; aux is set low and not used
Current sensor input	36	ADC1 channel 0, current transformer with 220 ohm burden
Main override switch / blast override switch	17 / 16	Active low, internal pull-up

A bulk capacitor of 470 uF or more from 3V3 to ground, close to the ESP32, is recommended: the PN532 self-test and RF start-up draw about 150 mA and marginal supplies cause start-up failures.

Tasks (FreeRTOS)

Task	File	Role
<code>task_led</code>	<code>led_ctrl.cpp</code>	Drives the four LEDs; flashes all twice at power-up
<code>task_rfid</code>	<code>rfid_task.cpp</code>	PN532 start-up and card polling; reads cards; sets the shared session state; handles config cards
<code>task_override</code>	<code>override_task.cpp</code>	Reads and debounces the two override switches (20 ms sampling, 3 samples)
<code>task_relay</code>	<code>relay_ctrl.cpp</code>	The only code that writes the relay pins; runs at 50 Hz
<code>task_session</code>	<code>session_task.cpp</code>	Counts starts and stops, accumulates machine run time and average current
<code>task_wifi</code>	<code>wifi_task.cpp</code>	Connects WiFi, sends INSERT and REMOVE messages, acts on the reply, posts diagnostics
<code>task_ota</code>	<code>ota_task.cpp</code>	Checks for and installs new firmware
current-sense task	<code>current_sense.cpp</code>	Samples the ADC through I2S DMA at 20 kHz on Core 1 and computes RMS current
diag heartbeat	<code>diag.cpp</code>	Records what the firmware was doing, for reboot diagnosis

Start-up order matters. `setup()` starts the current sensor, LED task, RFID task (which waits), override, relay and session tasks, then releases the RFID task so it can reset and initialise the PN532. WiFi is started only after the PN532 is initialised (up to 30 s wait), because radio noise during the reader's RF set-up causes intermittent failures on weak supplies. OTA starts last.

Shared state. All tasks share one `g_session` record (card active, member ID, authorized flag, start time, run time, average current, starts, stops, override flags) guarded by a mutex. Each hardware resource has one owner: the relay pins belong to `task_relay`, the PN532 to `task_rfid`, the override switches to `task_override`, and the ADC to the current-sense task.

Saved settings. The machine number (1 to 128; default 1) and blast-gate run-on time (default 30 s) are stored in flash (NVS, namespace `woodshop`) and change only through a signed config card.

Build modes. `DEV_BUILD` (commented out in `config.h`) disables the brown-out detector and stretches the watchdog for bench work. Leave it off for installed units so resets are logged rather than silent.

9. Client: cards, sessions, server veto, relays

Member card layout (v2, 52 bytes, unsigned)

Cards are NTAG215 tags written by `rfid_write.py` (companion to Lee Robertshaw's `WriteNTAG215.py`). The client reads them and never writes. Data starts at page 4.

Bytes	Field
0 to 3	Member ID as four ASCII digits, for example <code>0042</code>
4 to 19	Permission bitmap, 128 bits; bit 0 of byte 0 is machine 1
20 to 35	First name, ASCII, null-padded
36 to 51	Last name, ASCII, null-padded
52 onward	Zero

Member cards carry no signature, so the client trusts what is written on the card. Config cards (type byte 0x02, machine number, blast-gate delay in 10 s units, Ed25519 signature) are verified against the server's public key before they change the saved machine number or blast delay. A card whose first four bytes are not digits and is not a config card is rejected as unrecognised.

What happens when a card is presented

1. `task_rfid` polls the reader every 250 ms. On a new card it waits 150 ms, reads 120 bytes, and parses the member ID.
2. It checks the card's own permission bit for this machine (bit machine number minus 1). The result becomes the session's `authorized` flag, and the session is marked active.

3. If the card permits the machine: green LED on, yellow LED on. If not: green LED flashes fast and the machine stays off.
4. `task_relay` turns the main relay on at once when `active` and `authorized` are both true. This local grant works even if the server or network is down.
5. `task_wifi` notices the new session within 200 ms and sends an INSERT to the server.
6. If the reply status is non-zero, `task_wifi` clears `authorized` (the relay drops within one 20 ms tick) and marks the session as denied.
7. When the card has been absent for 500 ms, the session ends, all LEDs go off, and `task_wifi` sends a REMOVE with run time, average current, starts and stops. For a denied session it reports machine-on time, current, starts and stops as 0, but still sends how long the card was in.

If the server cannot be reached, nothing revokes the local grant and the machine keeps running (fail open).

Relays

Relay	On when
Main	(session active and authorized) or main override switch
Blast gate	1.5 s after the main relay turns on from a card (<code>BLAST_START_HOLD_MS</code>), or immediately with the main override; also while the blast override switch is on; also for the run-on time after the machine turns off

The 1.5 s hold-off lets the server's deny arrive before the blast gate starts, so a rejected card never runs the dust collector. The run-on timer (default 30 s, set by config card) starts only if the blast gate had actually been running.

Session statistics (`task_session`)

Every 50 ms while a card is present the task checks whether current is above 0.20 A (with hysteresis; it drops out at 80 percent of that). It counts starts and stops (each capped at 255), adds 50 ms to machine run time while the machine is on, and keeps a running mean of RMS current, ignoring the first 100 ms after each start to skip motor inrush.

10. Client: LEDs, sensing, overrides, updates, diagnostics

LED meanings

All four LEDs flash twice at power-up. Blink speeds: fast 400 ms period, medium 800 ms, slow 2 s.

LED	State	Meaning
Green	On	Card accepted locally; machine allowed
Green	Fast flash	Card denied, unreadable or unrecognised; also a rejected config card
Yellow	On	Card present
Yellow	Medium blink	No network (WiFi not connected); a card tap can briefly override it, and it re-asserts about every 5 s
Green and yellow	Fast flash for 2 s	Config card accepted
Red	Fast flash	Main override switch is on, or the card reader failed to start
White	Medium flash	Blast override switch is on
Red and white	Fast flash for 30 s	A firmware download or flash failed

Current sensing

A 1000:1 current transformer with a 10-turn primary and a 220 ohm burden resistor feeds GPIO36. The ESP32 samples it at 20 kHz through I2S DMA on Core 1 and keeps a fast RMS value and a slow DC-bias value. The machine counts as on above 0.20 A (idle noise is about 77 mA), with hysteresis so it does not chatter. The constants are in `config.h` (`CURRENT_*`).

Override switches

The two switches on GPIO17 (main) and GPIO16 (blast) bypass the card. The main override turns the main relay and blast gate on without a card; the blast override turns only the blast gate on. The state is sent to the server in the override byte of every message.

Firmware updates (OTA)

1. Build with Sketch > Export Compiled Binary. Copy it to the server's `firmware/` folder as `firmware.bin` and put the new version in `firmware/version.txt`. Raise `FW_VERSION` in `config.h` to match.
2. Run `python3 set_ota.py true`. It sets `UPDATE_AVAILABLE = True` in `master_server.py` and restarts the service, so every server reply carries the update flag.
3. Each client checks on its next card event, or on its own timer every 10 minutes, fetches `/firmware/manifest.json`, and does nothing if the version already matches its own.
4. If different, it downloads into the spare flash partition, checks the MD5, and reboots only after no card session is open, so power is never dropped mid-cut.

A client that is already current ignores the flag, so it can stay on for the whole rollout. Turn it off with `set_ota.py false` when done. Enabling bootloader rollback in the Arduino core is recommended so a bad image returns to the previous one.

Reboot diagnostics

At every boot the client records why it restarted (reset reason) and what it was doing just before (a breadcrumb kept in RTC memory), in a 16-entry ring buffer in flash. When WiFi is up it posts any pending record to `/diag/report` on the server, and the web app's Diagnostics page lists them. If most records show a brown-out or power-on reset, suspect the 3V3 supply rather than the code.

Changing machine number or blast delay

Present a signed config card to the machine. The server's private key signs these cards. The client checks the signature, the version byte (1) and the range (blast delay up to 150 s), then saves the values to flash.

11. Test and support tools

Tool	Runs on	Purpose
<code>BruceTestSender.py</code>	Any computer with Python 3	Reference sender for the kiosk format. <code>python BruceTestSender.py 192.168.0.5</code> sends 8 messages (4 logins, 4 logouts for members 403, 497, 9999 and 100, including a long-name truncation test), one second apart
<code>BruceTestSender_esp32.py</code>	ESP32 with MicroPython (Thonny)	The same test sequence ported to MicroPython: connects WiFi, syncs time over NTP, converts the 2000-based clock to Unix time, pads names without <code>ljust</code>
<code>rfid_write.py</code>	Raspberry Pi with a PN532 on SPI	Writes member cards. Example: <code>python3 rfid_write.py --id 42 --first Jane --last Doe --machines 1,3,5</code> . Accepts ranges (1-10,20), <code>all</code> or <code>none</code> ; member ID 0 to 9999; names up to 16 ASCII characters; <code>--dry-run</code> shows the bytes without a card; it verifies by reading back. Needs <code>adafruit-circuitpython-pn532</code> , Blinka and a GPIO library, and SPI enabled
<code>esp32_client.py</code>	ESP32 with MicroPython	Obsolete. Sends the earlier 52-byte ASCII format, which the server no longer accepts
<code>set_ota.py</code>	Server	Turns the firmware-update flag on or off and restarts the TCP service. Referenced in the code but not supplied for this document

The card writer's permission bitmap is 16 bytes, four little-endian 32-bit words, with machine 1 at bit 0. It matches the client's reader.

12. Open items and assumptions

#	Item	Detail
1	Machine numbering	Permission bits are 1-based (machine 1 is the first character). The server's <code>MACHINES</code> name table is 0-based, so machine 1 currently logs as "Band Saw". Confirm whether the controllers are numbered from 0 or 1 and fix the names or the bits to match
2	LEDs after a server denial	When the server vetoes a card, the relay drops but the green LED stays on and the yellow LED stays solid for the rest of the session. <code>rfid_task.cpp</code> comments describe a server-confirmation routine (<code>_attempt_server_confirmation()</code>) with a 30 s retry that is not in the current <code>wifi_task.cpp</code> . Decide whether to add a denial indication and the retry
3	Fail-open policy	If the server is unreachable the machine keeps running on the card's local grant. This is deliberate in the current design; say so if you want it changed to fail closed
4	Config cards	The client accepts only Ed25519-signed config cards, and its comments say the web app creates them at <code>/admin/config-card</code> . The <code>app.py</code> supplied has no such route, so how config cards are produced now is not documented here
5	Unsigned member cards	The client trusts the member ID and permissions written on a card. The server's own check (signed in, active, permitted) is the real gate
6	Server comments	The header comment of <code>master_server.py</code> still describes an older 16-byte layout with event-type and auth-status bytes. The code, and section 2 of this document, use the layout the client actually sends

#	Item	Detail
7	Services	The code names two systemd services, <code>woodshop</code> (web app) and <code>woodshop-tcp</code> (server). During testing the server was also started by hand. Confirm what starts each at boot
8	Default admin login	<code>admin / woodshop</code> is created if the credentials file is missing. Change it
9	Unused files	<code>current_sensor_task.c</code> and <code>current_sensor_dsp.c</code> are not referenced by the sketch; the build uses <code>current_sense.cpp</code> . <code>esp32_client.py</code> is obsolete
10	Not yet reviewed	<code>diag.cpp</code> , <code>ota_task.cpp</code> and <code>current_sense.cpp</code> were read for their header descriptions, not line by line

This document describes the code as supplied on the date above, including the changes made in this project: the 45-byte kiosk parser, kiosk rows in the web log, the live permission lookup with bit 0 = machine 1, inactive and expired members denied, the server-veto handling and 1.5 s blast hold-off in the client, the yellow-only no-network LED, the Refresh menu item, the daily log files and download page, and the daily NTP job.